

The Attention Scheduler

Interruption Economics as an Operating System Problem

P-RICK RESEARCH PROGRAM

an independent working-paper series on missing software

working paper P-006 v1.0 • series II

October 7, 2026 • cc by 4.0

Abstract — Operating systems schedule processors, memory, and network with fifty years of accumulated theory: admission control, queueing disciplines, priority inheritance, fairness guarantees, starvation freedom. The resource those machines exist to serve — human attention — is scheduled by a “Do Not Disturb” toggle and per-app volume knobs. The result is the defining failure mode of personal computing: an adversarial notification economy in which thousands of app vendors, optimizing for engagement, submit interruptions directly to the only queue that cannot be load-shed — a person’s working memory — at a measured cost of ~23 minutes of resumption lag per disrupted task, inflicted dozens of times per day, with no accounting, no admission control, and no audit trail. We argue attention is schedulable, and that the discipline needed to schedule it exists in pieces: interruption science (resumption-lag and breakpoint research) supplies the cost model; attention-sensitive notification research (Horvitz’s Bayesian work, Attelia’s breakpoint middleware) supplies the prediction machinery; modern OS focus features supply the user-legible policy vocabulary; on-device ML supplies the context model; and soft real-time scheduling theory supplies exactly the right formal shape — deadlines that may be missed, lateness that is priced, classes that must never starve. We specify the **attention scheduler**: an OS-level service with (1) an interruption-cost model over context signals, (2) admission control with deferral queues, (3) breakpoint-timed dispatch and batching, (4) per-app attention accounting and budgets, (5) priority hints with accountability (apps that cry urgent are charged for it), and (6) formal guarantees — bounded delivery latency for the urgent class, starvation-freedom for legitimate notifications, and stability against thrash. We design the architecture, define the formal properties, present the evaluation methodology (trace-driven simulation plus a field protocol), and confront the strongest objections: that context sensing is a privacy liability, that apps will game priority hints, that engagement-economics poison the platform, and that users do not want machines deciding for them. The timing argument is uncomfortable: Apple Intelligence’s “Reduce Interruptions” and Android’s notification cooldowns are the first production steps toward exactly this system — built closed, unaccountable, and unexplainable. The research gap is thus not “will it happen” but “will it have a theory”: without published scheduling semantics, attention management will become another opaque engagement lever. The paper supplies that theory before the closure.

p-rick working paper P-006 · series II (systems) · draft 1.0

Operating systems schedule processors, memory, and network with fifty years of accumulated theory: admission control, queueing disciplines, priority inheritance, fairness guarantees, starvation freedom. The resource those machines exist to serve — human attention — is scheduled by a “Do Not Disturb” toggle and per-app volume knobs. The result is the defining failure mode of personal computing: an adversarial notification economy in which thousands of app vendors, optimizing for engagement, submit interruptions directly to the only queue that cannot be load-shed — a person’s working memory — at a measured cost of ~23 minutes of resumption lag per disrupted task, inflicted dozens of times per day, with no accounting, no admission control, and no audit trail. We argue attention is schedulable, and that the discipline needed to schedule it exists in pieces: interruption science (resumption-lag and breakpoint research) supplies the cost model; attention-sensitive notification research (Horvitz’s Bayesian work, Attelia’s breakpoint middleware) supplies the prediction machinery; modern OS focus features supply the user-legible policy vocabulary;

on-device ML supplies the context model; and soft real-time scheduling theory supplies exactly the right formal shape — deadlines that may be missed, lateness that is priced, classes that must never starve. We specify the **attention scheduler**: an OS-level service with (1) an interruption-cost model over context signals, (2) admission control with deferral queues, (3) breakpoint-timed dispatch and batching, (4) per-app attention accounting and budgets, (5) priority hints with accountability (apps that cry urgent are charged for it), and (6) formal guarantees — bounded delivery latency for the urgent class, starvation-freedom for legitimate notifications, and stability against thrash. We design the architecture, define the formal properties, present the evaluation methodology (trace-driven simulation plus a field protocol), and confront the strongest objections: that context sensing is a privacy liability, that apps will game priority hints, that engagement-economics poison the platform, and that users do not want machines deciding for them. The timing argument is uncomfortable: Apple Intelligence’s “Reduce Interruptions” and Android’s notification cooldowns are the first production steps toward exactly this system — built closed, unaccountable, and unexplainable. The research gap is thus not “will it happen” but “will it have a theory”: without published scheduling semantics, attention management will become another opaque engagement lever. The paper supplies that theory before the closure.

1 Introduction

A modern smartphone user receives on the order of 50–100 notifications per day; heavy users receive hundreds. Each one is a scheduling decision made by an app vendor’s growth team and executed on a human’s working memory without consulting anything except a static allow/deny list the user last touched months ago. The interruption-science literature has measured the consequences for two decades: disrupted knowledge workers take ~23 minutes on average to fully resume (Mark et al., 2008), resumption quality degrades with interruption cost (Altmann & Trafton’s memory-for-goals theory), self-reported stress rises with notification volume, and the “attention span” literature documents the collapse of uninterrupted focus blocks from ~2.5 minutes mean in 2004 to under a minute by the early 2020s (Mark’s longitudinal line).

The systems response, to date, is a filter. Do Not Disturb, Focus modes, iOS Scheduled Summary, Android channels and cooldowns, Windows Focus Assist — every shipped feature is a static or rule-based *admission filter*, decided per-notification at arrival, with no memory, no cost model, no queueing, and no accounting. The filter vocabulary (“allow,” “silence,” “deliver quietly,” “batch at 9 AM”) has no notion of *when the human is cheap to interrupt* — the single variable interruption science identifies as dominant. Breakpoint research (Iqbal & Bailey’s OASIS line) showed that delivering the *same* notifications at task breakpoints cuts measured interruption cost dramatically; Attelia (Okoshi et al.) detected breakpoints from wearable/phone sensors and demonstrated double-digit reductions in perceived interruption load in field deployments. The prediction machinery works. It has never been given an operating system to live in.

This paper’s claim is architectural: **the missing artifact is a scheduler, and schedulers are specified by their guarantees.** A filter asks “is this notification allowed?” A scheduler asks — and answers — “what is the cheapest feasible moment in the next N minutes to deliver this, given a cost model, a queue, deadlines, and a budget?” The difference is exactly the difference between a firewall and a CPU scheduler, and it is the difference the notification stack has never crossed.

2 The problem: attention as an unscheduled resource

Formalize the current architecture. Apps submit notifications at times of *their* choosing. The OS applies static filters and ranking heuristics, then *immediately renders* whatever survives (or defers everything to a digest — the one degenerate scheduler users have). There is no:

- **Cost model.** Nothing estimates what an interruption at time t costs the user — despite the literature supplying the predictive variables (task engagement, resumption state, calendar state, motion, recent interaction patterns).
- **Admission control.** Nothing decides *whether* the marginal notification is worth its cost at all, per app, per day. The only admission control ever fielded is user rage (disabling notifications app by app, at high abandonment cost to legitimate apps).
- **Queueing discipline.** Deferred notifications have no policy: iOS digests batch by clock time, not by breakpoint or batching *size* research (which shows batching reduces cost but delays degrade usefulness — a real queueing tradeoff with published coefficients).
- **Accounting.** No per-app ledger of interruptions delivered, cost inflicted, latency incurred. The user cannot see that app X cost them 47 interruptions this week; the platform cannot price priority hints; no one can audit the engagement economy. (Screen-time dashboards count minutes, not interruptions-by-cost.)
- **Guarantees.** Nothing is promised to anyone: not to the user (an urgent message will arrive within T), not to apps (well-behaved notifications will not starve), not to regulators (a verifiable policy of what was suppressed and why). The urgent-message starvation failure of aggressive filtering — the reason users distrust automation and toggle it off — is a *scheduling* bug: no deadline class existed, so a filter had no way to be wrong gracefully.

Two decades of HCI research has treated these as interface problems. The contribution of this paper is to treat them as one systems problem with one service boundary: an OS-level **attention broker** that owns the queue, the cost model, the budget accounting, and the deadline classes, exposing three interfaces — to apps (a priority-hint API with honesty pricing), to the user (policy, ledgers, and explainability), and to sensors (context signals with local processing).

3 The design space: what exists, and the empty center

Interruption science (the cost model). Mark, Gudith & Klocke (CHI 2008) measured the ~23-minute resumption cost and the paradox that interrupted workers work *faster but more stressed*. Altmann & Trafton’s memory-for-goals theory (2002–2004) gives the resumption mechanism (priming decay) and predicts resumption-lag structure. Iqbal & Bailey’s line (CHI 2005; UIST 2005 — OASIS) identified *task breakpoints* as the low-cost moments and built the first notification-delivery framework around them. Balivet? no — the complement: Adamczyk & Bailey (CHI 2004) showed breakpoint-timed delivery reduces frustration and workload in controlled studies.

Attention-sensitive computing (the prediction machinery). Horvitz’s Microsoft line — Lumiere (2000), attention-sensitive alerting (1999), “Priorities” (CHI 2003) — built Bayesian models of user attention and cost-of-interruption for messaging, demonstrating learning-based best-time-to-interrupt years before the hardware could deploy it ubiquitously. Okoshi et al.’s Attelia (2015–2017) detected breakpoints from phone/wearable sensor streams in the wild and scheduled notifications to them, with measured reductions in perceived load — the strongest field evidence that breakpoint dispatch works on commodity hardware. The JITAI literature (just-in-time adaptive interventions, health informatics) applies the same timing question to therapeutic notifications.

Production OS features (the policy vocabulary, closed). iOS: Focus modes (2021), Scheduled Summary (2021, clock-batching), Apple Intelligence “Reduce Interruptions” (iOS 18.1, 2024 — on-device

importance-understood delivery) and priority notifications (2025). Android: notification channels (2017), Adaptive Notifications (2019-10, on-device ranking), notification cooldown (Android 15, 2024 — rate-limiting *repeated* alerts), Modes (15/16). Windows/macOS: Focus Assist/sessions, Focus modes. Every one of these is a *filter or ranker*: none exposes deadlines, budgets, accounting, or explainability; all are closed models the user cannot audit. BlackBerry’s Priority Hub (2013) deserves the ancestor citation — learned per-sender importance, no scheduling semantics.

Notification-management apps (the demand evidence). Daywise (batching), BuzzKill and its rule-engines, the summarizer wave — each validates user demand for *control* and dies against the platform’s closed APIs: an app cannot see the notification stream, cannot delay peer notifications, cannot read context sensors freely. The demand is proven; the platform boundary prevents the product.

Scheduling theory (the shape of the answer). Soft real-time scheduling (deadlines with priced misses), EDF and its tardiness bounds, queueing with admission control under overload, fair queueing and starvation-freedom — a half-century of machinery for resources that cannot serve everyone at once. The literature that applies it to *humans as the resource* is nearly empty: scattered position papers and the JI-TAI heuristic literature, no scheduler specification with formal properties. That absence — not algorithmic difficulty — is the gap.

The center is empty in a specific way: **no system has ever shipped an interruption queue with a cost model, deadlines, budgets, and an audit ledger.** The ingredients are individually proven; the composition has never been specified or built, and is now being approached blindly by closed platform features.

4 Why the gap persists

The engagement-economy misalignment. App vendors profit from interruption; the platform profits from app ecosystems; the user pays the cost. Any attention scheduler that exposes accounting threatens the economics — which is precisely why none of the closed features ship ledgers. This is a *political* barrier, not technical, and it explains why the honest version of this system must come with regulation-adjacent justification (§8) or from a neutral platform.

Context sensing is privacy-sensitive. A cost model is only as good as its signals (calendar, sensors, interaction history), and all of them are surveillance-grade if mishandled. The barrier expired on-device: the model can run locally, on signals already granted to the OS, with no network egress — but the *design discipline* (local-only, explainable, inspectable) must be architectural, not promised.

The evaluation gap. Interruption cost is subjective and slow to measure (diary studies, experience sampling), so the field never developed the trace-driven evaluation culture that made CPU schedulers improvable. The fix: instrumented field studies producing anonymized interruption traces (interaction logs + ESM labels), then offline simulation on those traces — the methodology this paper proposes (§7) as the community standard.

Users fear opaque automation. Aggressive filtering that eats one urgent message converts a user against automation forever (one-shot trust destruction). Filters earned this fear honestly. Schedulers repair it the way schedulers always have: with *guarantees* — the urgent class has a bounded-latency promise, everything else has a starvation-freedom promise, and every suppression has an explanation in the ledger. Trust follows from auditable properties, which filters cannot offer.

5 The attention scheduler: specification

5.1 The object model

Notifications become **attention requests** $R = (\text{app}, \text{class}, \text{payload}, \text{submit-t}, \text{deadline}, \text{priority-hint})$. Classes are a small closed set: **urgent** (person-to-person, time-critical: calls, safety alerts, two-factor codes), **interactive** (conversation turns), **informational** (content updates), **ambient** (presence of new stuff). Deadlines: urgent requests carry $d \in [30s, 2m]$ hard-ish; interactive $d \in [5m, 1h]$ soft; informational $d \in [1h, \infty)$; ambient unbounded (delivered by pull or next dispatch window). Priority hints are *app-claimed* importance, priced by §5.4.

5.2 The cost model

$C(t)$ = expected cost of an interruption at time t , computed locally from: current task engagement signals (interaction continuity, keystroke/scroll rhythm, window focus), breakpoint proximity (the Attelia-style detector as a service), calendar state (meeting vs. focus block), motion/setting (driving = hard suppression for all but urgent), historical response patterns per class and per app, and time-of-day priors. Output is a cost curve over a forward horizon — “now is expensive; a cheap window is predicted in ~4 minutes at a task boundary” — precisely the shape OASIS’s results motivate. The model is local, versioned, and inspectable: its feature list and weights are exposable to the user (explainability requirement, §5.6).

5.3 The dispatch policy

At each evaluation instant (cost-curve update, new request, deadline approach), the broker solves a small online scheduling problem: deliver now, or defer to the next predicted low-cost window, subject to (a) class deadlines, (b) batching economics (batch size $\leq k$, per the batching literature), (c) budget state (§5.4), (d) starvation constraints. The policy family is deliberately conservative and analyzable: earliest-deadline-first with cost-aware defer windows and a hard floor that urgent requests pre-empt everything (rendered through the safety channel — full-screen/ring). Interactive requests defer to the nearest breakpoint within their deadline; informational requests batch into dispatch windows aligned to breakpoints or calendar boundaries; ambient never interrupts (pull only). Every decision is ledgered.

5.4 Accounting and honesty pricing

Each app holds an attention budget (default daily cap per class, user-adjustable). Delivered interruptions *charge* the budget at realized cost (did the user act on it? engage? dismiss within 2s?). Apps that submit honest priority hints — hints that, post-hoc, match user-assessed importance (rated via sparse experience sampling and behavioral proxies) — earn cheaper priority; apps repeatedly crying urgent for content updates pay escalating costs until their urgent-class submissions are rate-limited. This is the mechanism the engagement economy has never had: **the marginal notification from a low-value app becomes expensive, in a currency the platform enforces**. The ledger per app (delivered, deferred, suppressed, realized cost, deadline violations) is user-facing and exportable — the audit surface regulators will eventually demand and that §8 argues is the political unlock.

5.5 Formal properties

Three guarantees, stated in the style of real-time scheduling theory:

- **Urgent bounded latency:** for the urgent class, $P(d \text{ lateness} > \epsilon) \leq \delta$ — deliveries exceed deadline by more than ϵ with probability at most δ , under an admission model of urgent-arrival rate (the safety class is capacity-reserved; spam in it is a policy violation, priced by 5.4).

- **Starvation freedom:** every admitted non-urgent request is delivered within $\max(\text{deadline}, W_{\text{max}})$ — W_{max} a user-set worst-case wait (default: end of day) — so automation can never silently eat a message; the failure mode of filters becomes structurally impossible.
- **Stability/thrash bounds:** at most B batch-dispatch events per hour; no more than one urgent pre-emption per M minutes absent true emergencies — bounds against the degenerate solutions (dozens of micro-batches; urgent-storm abuse).

These are modest, checkable properties — the same style as forward-progress guarantees in schedulers and databases. They are what convert “smart notification filtering” from a trust-me product into an auditable service.

5.6 Architecture

OS attention broker: (1) **request API** for apps (submission with class + hint; the current notification API is a compatible subset — unmodified apps submit as informational with default hints); (2) **context service** fusing local signals into the cost curve (no egress; model local; versioned and diffable per OS update); (3) **policy engine** implementing 5.3 with properties 5.5 enforced by construction and runtime-monitored; (4) **ledger** (append-only, user-owned, exportable) recording every decision with reason codes — the explainability substrate; (5) **user surfaces:** budget console, daily attention statement (“46 interruptions; est. 71 min of focus protected”), per-app ledgers, and a policy DSL in the familiar focus-mode grammar but scheduler-semantic (“urgent always within 2m; conversations at breakpoints; everything else at lunch and 6pm; cap app X at 3/day”).

6 Evaluation design

Two-stage, trace-first — the methodological contribution the field lacks:

Stage 1 — simulation on real traces. Instrumented field collection ($n \approx 40$, 2 weeks): interaction logs (window focus, input events at coarse granularity), delivered notifications with class/app/label, experience-sampling prompts at breakpoints and interruptions (cost self-ratings), sensor context. Traces yield: cost-model offline validation (predicting ESM cost from signals), then scheduler replay — current-OS delivery vs. attention-scheduler delivery on identical request streams, measuring: interruption cost proxy (ESM-validated), delivery lateness distributions per class, starvation events (must be zero by construction), deferral accuracy (were predicted breakpoints actually cheap?). Baselines: immediate delivery (status quo), static Focus schedule, digest-batching.

Stage 2 — living-OS field deployment. A rooted/research-build Android or a desktop environment implementing the broker natively, $n \approx 20$, 4-week A/B crossover: measures trust (automation on/off rates — the honest primary endpoint), self-reported interruption load (validated scales), urgent-class lateness in the wild, and app-budget effects (does informational volume from apps *decrease* when charged? — the economics question).

7 Objections, confronted

“Context sensing is surveillance.” The design is local-only, and the ledger is the proof: no egress path exists for the model, and the feature list is inspectable. The honest concession: local models still leak through their *decisions* (what gets deferred reveals what you were doing), so the ledger must be device-local and encrypted, and the research build should publish traffic-analysis results. The principle: attention data must be *less* exposed than the data the notifications themselves already carry.

“Apps will game the hints.” That is the point of pricing: gaming is *priced in*. An app that mislabels content as urgent pays escalating costs and loses urgent-class access — the failure state of the current system (everything screams) becomes expensive instead of free. The concession: importance ground truth is only knowable post-hoc and noisily; the pricing model must be tolerant (honest apps are not bankrupted by label noise), so costs escalate on *persistent* dishonesty statistics, not single events.

“Users don’t want machines deciding.” Users don’t want *unauditable* machines deciding — the evidence is their relationship with every existing smart feature: adopted when explainable and reversible (spam filters, focus modes), rage-disabled after the first silent failure. The scheduler’s properties (starvation-freedom, urgent-bounded-latency, statement surfaces) are engineered for one-shot-failure immunity. The residual risk is real: bad cost models defer the wrong things and users never re-enable. Hence trace-first evaluation and conservative defaults (interactive-heavy users get near-immediate delivery until the model proves itself).

“Platforms will never ship honesty pricing.” Apple’s Reduce Interruptions shows they will ship the *model*; they will resist the *ledger*. The open path: desktop Linux and research builds demonstrate the semantics; regulators (EU digital-wellbeing and dark-pattern lines) eventually demand exactly the audit surface this design defines; the specification — not any one vendor’s implementation — is the durable contribution. This is the same trajectory privacy nutrition labels took: research artifact → regulation vocabulary → product feature.

“Attention is not schedulable — costs are idiosyncratic.” The strongest theoretical objection. Answer: scheduling theory never needed exact job durations — it needed *orderable* costs under uncertainty, which is what the interruption literature provides (breakpoints < mid-task; meetings < flow; driving < everything). The scheduler is explicitly a decision-under-uncertainty machine with robustness properties, not an oracle. Where individual costs diverge from priors, budgets and policies are the user’s handles on the model — the system is tunable in the dimension where it is fallible.

8 The AI era makes this more necessary, not less

The notification stream is about to be overwhelmed by agents. Personal AI assistants, coding agents, monitoring bots, and workflow automations are notification *emitters* — and they emit at machine rate, with the engagement economy’s incentives replaced by something worse: an agent’s own convenience. A human overseeing five agents that each “ping when they need input” is a human with five new un-throttled queues attached to their working memory. The attention scheduler is the missing host-side admission layer for agentic computing: agents submit attention requests with class and hints; the broker’s accounting makes agent interruption *cost-visible* (the “agent attention bill”), the deadline classes give agents a contract to code against (submit with $d=1h$, receive delivery guarantee), and the ledger makes agent behavior auditable. The platforms are moving (Apple’s priority notifications, Gemini’s awareness lines) but closed; the specification this paper offers is the open alternative, and the window to define the semantics before three closed implementations define them by default is, at best, two or three years.

9 Roadmap

Phase 1: cost-model evaluation on existing public interaction datasets; scheduler specification as a formal technical report; open trace format and simulator released. Phase 2: instrumented field study (Stage 1 of §6); first cost-model checkpoints published as on-device-sized artifacts. Phase 3: research-OS deployment (Stage 2); agent-request API proposal to one agent framework; policy DSL standardization conversation. Open problems: cost-model transfer across users (personalization vs. cold start); batching-optimal dispatch under deadline uncertainty (a genuinely open online-scheduling problem); cross-device attention state (phone + watch + laptop as one queue — deferred to P-007’s fabric); regulatory semantics of the ledger (what a “suppressed, reason: budget” record means in consumer-protection law).

10 Conclusion

Half a century of systems research has produced something remarkable: a body of theory that lets a shared machine serve thousands of competing requesters with fairness, latency guarantees, and auditability — while starving none. The resource that theory was *for* has never been given the machinery. Human attention is scheduled today by an adversary’s growth team and a toggle switch, at a measured cost no one records, with guarantees no one can audit. The literature has built every piece — the cost model, the breakpoint detector, the budget insight, the platform vocabulary — and left them unassembled. This paper assembles them into a scheduler with properties: urgent bounded latency, starvation freedom, honesty pricing, an attention ledger. The uncomfortable closing observation is that the closed platforms have begun building exactly this, without the properties, without the ledger, and without the theory. Either the research field specifies attention scheduling now, or it will be specified by three product managers, in private, as one more engagement lever. The queue is open. Someone should schedule it.

Disclosure

p-rick is an independent research program; no vendor has commissioned or reviewed this work; the author has no financial interest in cited systems. This paper shares its series-II framing with P-004 (degradation contracts) and P-007 (the compute fabric) but no product overlap; cross-references are flagged in text.

References

1. Mark, G., Gudith, D., & Klocke, U. (2008). *The cost of interrupted work: more speed and stress*. ACM CHI.
2. Mark, G., et al. (2023). *Attention Span* (book) and the UCI longitudinal attention line (2004–2024).
3. Altmann, E. M., & Trafton, J. G. (2002). *Memory for goals: an activation-based model*. Cognitive Science 26(1).
4. Iqbal, S. T., & Bailey, B. P. (2005). *Investigating the effectiveness of various types of notifications for task interruption and resumption* — and OASIS framework line, UIST 2005.
5. Adamczyk, P. D., & Bailey, B. P. (2004). *If not now, when? The effects of interruption at different moments within task execution*. CHI 2004 (breakpoint-timed delivery evidence).
6. Horvitz, E. (1999/2003). *Principles of mixed-initiative user interfaces*; the Lumiere line (2000); Horvitz & Apacible, learning and reasoning about interruption; “Priorities” (CHI 2003).
7. Okoshi, T., et al. (2015–2017). *Attelia* line: sensing and delivering notifications at breakpoints (Ubi-Comp/IMWUT venues).
8. Nahum-Shani, I., et al. (2018). *Just-in-time adaptive interventions (JITAI): a framework for health behavior research*. Translational Behavioral Medicine.

9. Abel, F., Kreitz, G., & Nord, T. (2020). *Attention aware computing* survey line, ACM/Now Foundations.
10. Simon, H. A. (1971). *Designing organizations for an information-rich world* (“a wealth of information creates a poverty of attention”).
11. Weiser, M., & Brown, J. S. (1996). *The coming age of calm technology*. Xerox PARC.
12. Apple iOS documentation (2021–2025): Focus, Scheduled Summary, Apple Intelligence Reduce Interruptions (iOS 18.1), priority notifications (iOS 18.4+).
13. Android documentation (2017–2025): notification channels, Adaptive Notifications (Android 10), notification cooldown (Android 15), Modes.
14. BlackBerry 10 (2013). *Priority Hub* — learned per-sender importance (the production ancestor).
15. Liu, C. L., & Layland, J. W. (1973). *Scheduling algorithms for multiprogramming in a hard-real-time environment* — the formal ancestry, cited for the *shape* of guarantees, not the domain.
16. Buttazzo, G., et al. soft real-time scheduling and tardiness bounds literature (the lateness-priced model).
17. Pielot, M., et al. (2014). *When attention is not scarce — detecting boredom from mobile phone usage* (field-scale notification/attention measurement).
18. Daywise (acquired product, batching), and the notification-management app ecosystem (demand evidence).
19. Fisher, M., et al. regulatory literature on digital wellbeing and dark patterns (EU consumer-protection line, 2022–2025).
20. Okoshi, T., et al. (2014–2019). The *reducing notification burden* line: importance tokenization and notification scheduling, Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies (IMWUT).